Review :

“ Design of Controller for three-phase Grid-Connected Inverter Using Reinforcement Learning ”

（強化学習を用いた三相系統連系インバータ用制御器の設計）

Author : Raj kumar Mahto, Pankaj Mishra, Aftab Alam

(ISTEMS 2024)

資料作成：AE22042 福万恭平

1. 研究概要

■研究内容

電圧形インバータ(VSI)向けの、より高性能な新しい制御器設計の提案

■従来制御の課題

従来のPI制御、PR制御、DB制御、MPCなどには構造的な限界があった

- モデルへの強依存 → 環境変動や経年劣化などでシステム条件が変わると、固定された制御ゲインでは対応しきれずに性能低下
- 動的性能と定常性能のトレードオフ → 応答を早くしようとすると、オーバーシュートが発生
- 調整の複雑さ → 上記トレードオフの中で、最適な制御器ゲインの調整作業が複雑かつ困難

■論文の解決アプローチとその結果

RL-DDPGによる強化学習制御を利用し、従来制御より良い制御をすることができた

- ➢全高調波歪(THD)を約47%低減
- ➢オーバーシュートを8%に抑える
- ➢定常誤差・リップルを最小化
- ➢過渡期間の目標値の変動を低減

2. 提案手法内容　①強化学習前の下処理(d-q変換)

■d-q変換の目的

強化学習に使うパラメータ数を削減し、
制御計算をより簡素にする

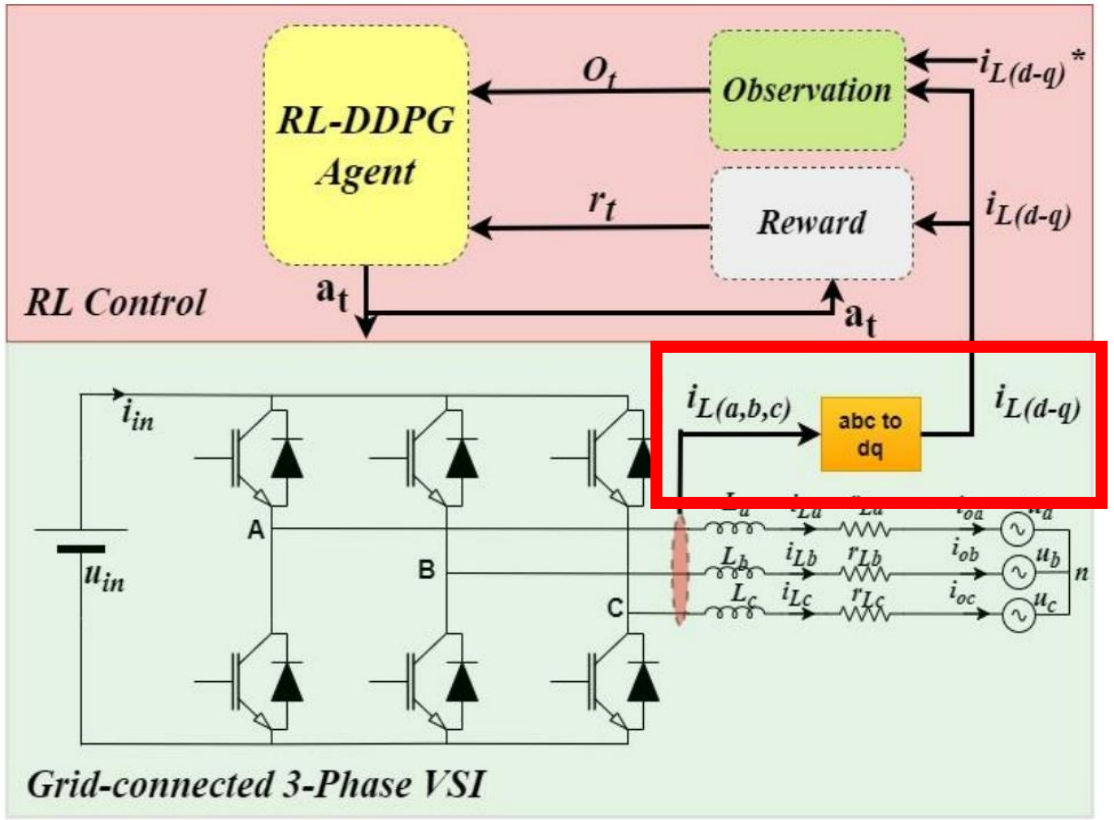


Fig. 1 Architecture of Reinforcement Learning control

3相座標系(固定軸)

$$\langle u_{La}\rangle = d_A\langle u_{in}\rangle - r_{eq}\langle i_{La}\rangle - \langle u_{an}\rangle - \langle u_{nN}\rangle \quad (1)$$

$$\langle u_{Lb}\rangle = d_B\langle u_{in}\rangle - r_{eq}\langle i_{Lb}\rangle - \langle u_{bn}\rangle - \langle u_{nN}\rangle \quad (2)$$

$$\langle u_{Lc}\rangle = d_B\langle u_{in}\rangle - r_{eq}\langle i_{Lc}\rangle - \langle u_{cn}\rangle - \langle u_{nN}\rangle \quad (3)$$

▼

α-β座標系(固定軸)

$$\frac{d\langle \boldsymbol{i}_L\rangle}{dt} = \frac{1}{\boldsymbol{L}}\left[\boldsymbol{d}\langle u_{in}\rangle - r_{eq}\langle \boldsymbol{i}_L\rangle - \langle \boldsymbol{u_0}\rangle\right] \quad (9)$$

▼

d-q座標系(回転軸)

$$\frac{d\langle i_{Ld}\rangle}{dt} = \frac{1}{L}\left[d_d\langle u_{in}\rangle - r_{eq}\langle i_{Ld}\rangle + \omega_s L\langle i_{Lq}\rangle - \langle u_{od}\rangle\right] \quad (15)$$

$$\frac{d\langle i_{Lq}\rangle}{dt} = \frac{1}{L}\left[d_q\langle u_{in}\rangle - r_{eq}\langle i_{Lq}\rangle - \omega_s L\langle i_{Ld}\rangle - \langle u_{od}\rangle\right] \quad (16)$$

2. 提案手法内容　①強化学習前の下処理(小信号モデル)

■小信号モデルとは

d-q変換で得られた数式に含まれる非線形要素を除去し、線形モデルへ変換すること

(例)一定値と微変動値に分けた変数

$$d_d = D_d + \hat{d}_d \quad と \quad \langle u_{in} \rangle = U_{in} + \hat{u}_{in}$$

を掛け合わせて線形項を抽出

$$\frac{d\langle i_{Ld}\rangle}{dt} = \frac{1}{L}\left[d_d\langle u_{in}\rangle - r_{eq}\langle i_{Ld}\rangle + \omega_s L\langle i_{Lq}\rangle - \langle u_{od}\rangle\right] \qquad (15)$$

$$(D_d + \hat{d}_d)(U_{in} + \hat{u}_{in}) = \cancel{D_d U_{in}} + \boxed{D_d\hat{u}_{in} + \hat{d}_d U_{in}} + \cancel{\hat{d}_d\hat{u}_{in}}$$

定常値　線形項　微小項

■小信号モデルにより得られる状態方程式

$$\frac{d\hat{i}_{Ld}}{dt} = \frac{1}{L}[\hat{d}_d U_{in} + D_d\hat{u}_{in} - r_{eq}\hat{i}_{Lq} + \omega_s L\hat{i}_{Ld} - \hat{u}_{od}] \qquad (20)$$

$$\frac{d\hat{i}_{Lq}}{dt} = \frac{1}{L}[\hat{d}_q U_{in} + D_q\hat{u}_{in} - r_{eq}\hat{i}_{Lq} - \omega_s L\hat{i}_{Ld} - \hat{u}_{oq}] \qquad (21)$$

$$\hat{i}_{od} = \hat{i}_{Lq} \qquad (22)$$

$$\hat{i}_{oq} = \hat{i}_{Lq} \qquad (23)$$

▼ 状態方程式へ

$$\hat{x}(t) = \begin{bmatrix}\boxed{\hat{i}_{Ld} \quad \hat{i}_{Lq}}\end{bmatrix}^T$$

$$\hat{u}(t) = \begin{bmatrix}\hat{u}_{in} & \hat{u}_{od} & \hat{u}_{oq} & \boxed{\hat{d}_d \quad \hat{d}_q}\end{bmatrix}^T$$

$$\hat{y}(t) = \begin{bmatrix}\hat{i}_{od} & \hat{i}_{oq}\end{bmatrix}^T$$

$$\left\{\begin{aligned}\frac{d\hat{x}(t)}{dt} &= A\hat{x}(t) + B\hat{u}(t)\\ \hat{y}(t) &= C\hat{x}(t) + D\hat{u}(t)\end{aligned}\right\}$$

$$A = \begin{bmatrix}\frac{-r_{eq}}{L} & \omega_s\\ -\omega_s & \frac{-r_{eq}}{L}\end{bmatrix}, \quad B = \begin{bmatrix}\frac{D_d}{L} & \frac{-1}{L} & 0 & \frac{-U_{in}}{L} & 0\\ \frac{D_q}{L} & 0 & \frac{-1}{L} & 0 & \frac{-U_{in}}{L}\end{bmatrix}$$

$$C = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}, \quad D = \begin{bmatrix}0 & 0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0 & 0\end{bmatrix}$$

2. 提案手法内容　②強化学習のフレームワーク

■強化学習

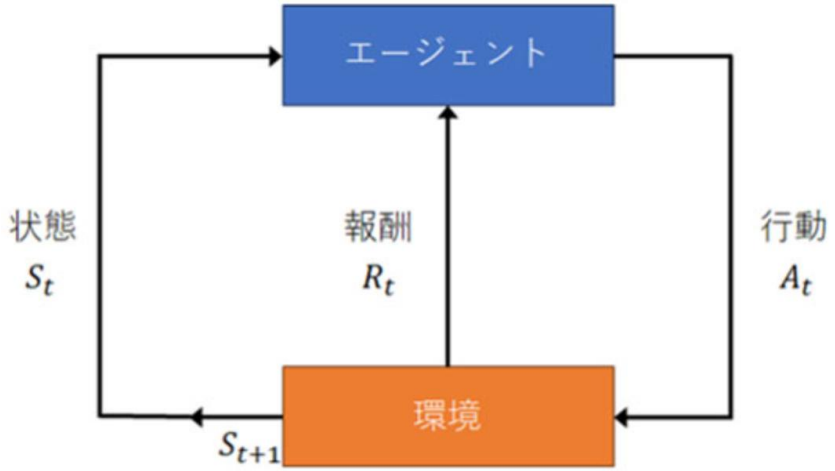
✓強化学習のゴール = 報酬をできる限り大きくする方策を見つけること

✓本論文ではマルコフ決定過程の枠組みを利用

※方策：エージェントが従う行動のルール

■マルコフ決定過程(MDP)

状態遷移が「現在の状態」と「そのときの行動」によって決定するモデル



(https://www.geolab.jp/documents/column/ai-012/)

■MDPを利用したワークフロー

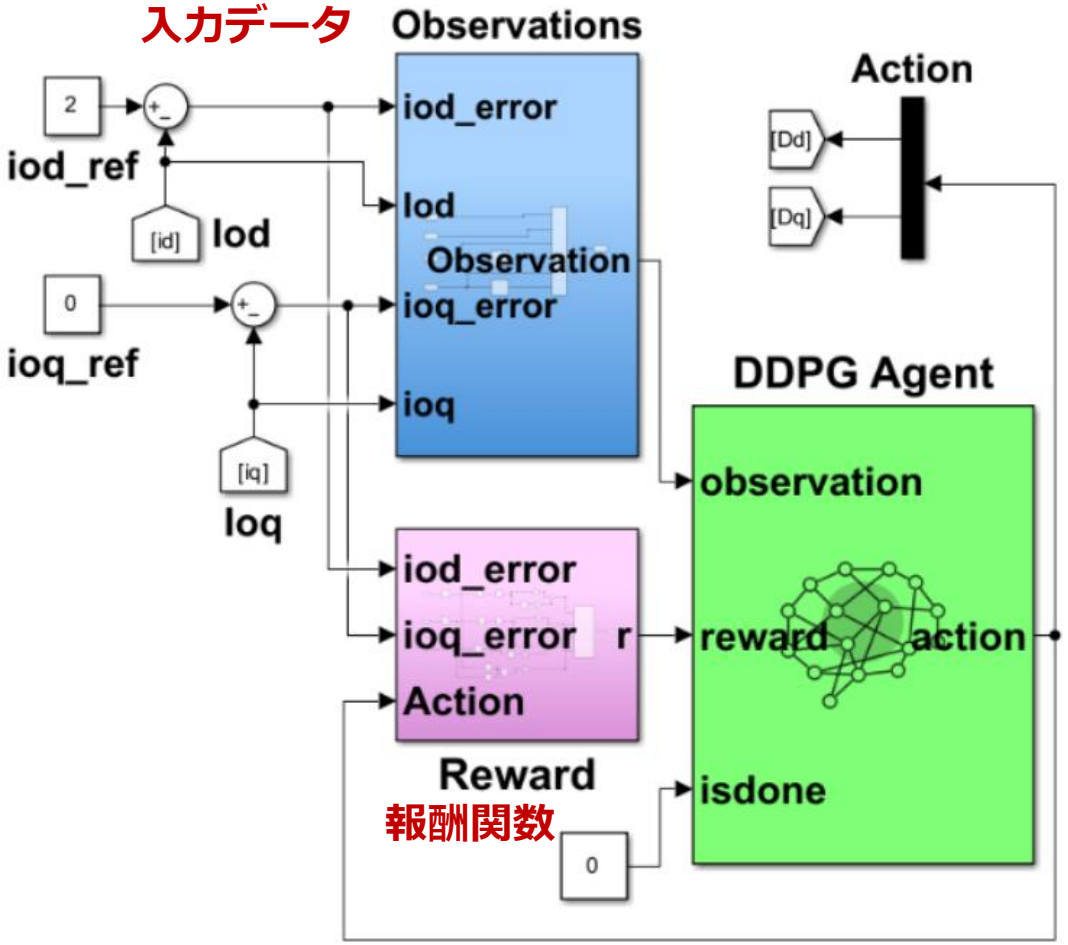


Fig. 2 Implementation of the DDPG agent in Simulink

2. 提案手法内容　②強化学習のフレームワーク

■報酬関数(Reward)

- AIのアクションを評価する採点基準
- 重み付け係数 p_1, p_2, p_3 でバランス調整

$$r = -\left(p_1 i_{derror}^2 + p_2 i_{qerror}^2 + p_3 \sum (u_{t-1})^2\right) \quad (30)$$

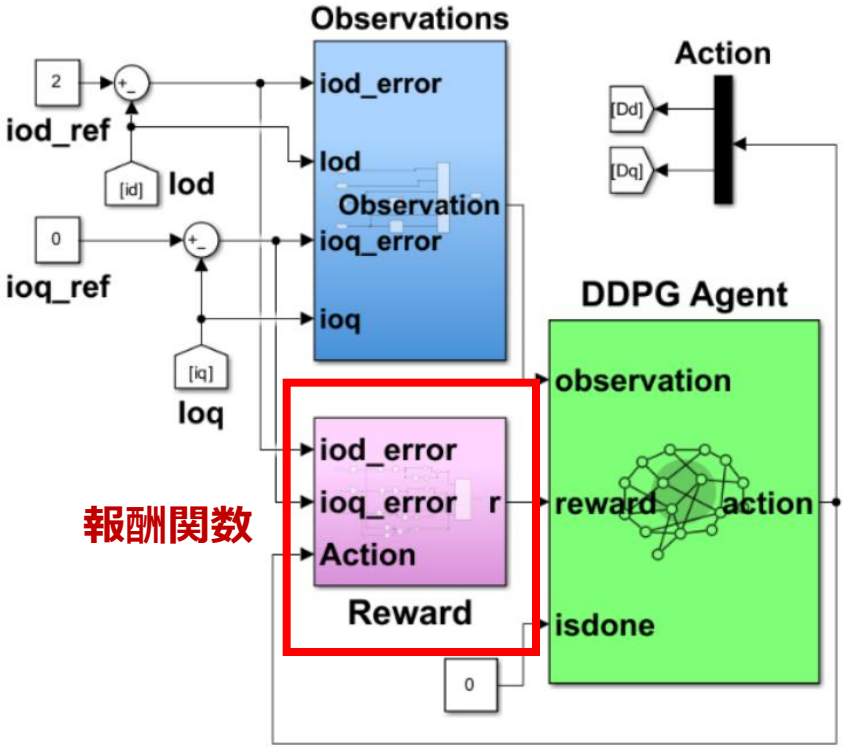


Fig. 2 Implementation of the DDPG agent in Simulink

■DDPGアルゴリズム

- 2つの深層NN「アクター」と「クリティック」が対話するように働く
- 下図の過程で合計報酬を最大化する最適な方策を探して見つける

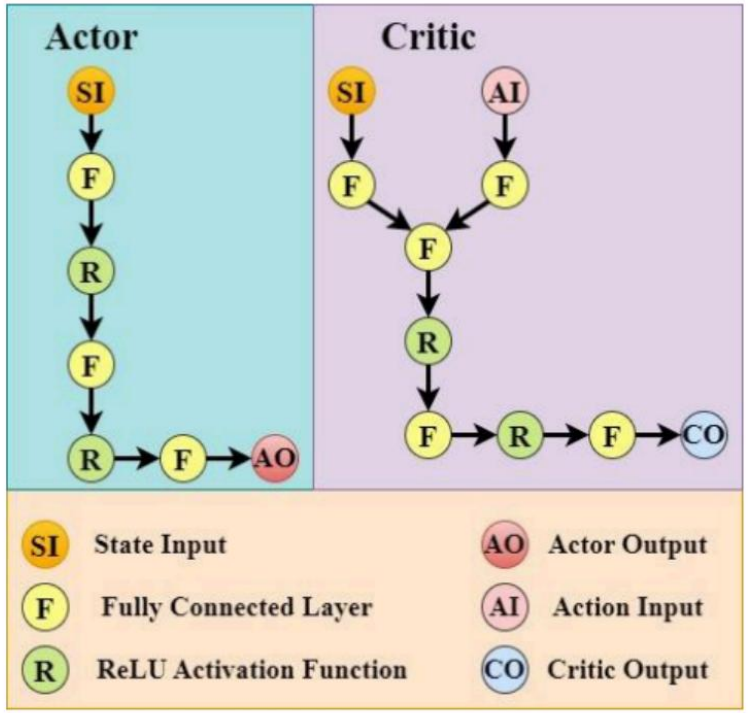


Fig.4 Configuration of Actor-Critic Deep Neural Network

3. シミュレーション結果

■DDPGのパフォーマンス

(今回の訓練では900点が学習完了のゴール)

✓約250回の練習で目標値を達成

✓達成後もスコアを高い位置でキープ

→ 試行錯誤の繰り返しで、最適コントロールが可能である

■THD(ノイズ)品質

✓PI制御：1.41%

✓DDPG：0.74%

→ AI制御で、PI制御より高品質で安定した電力を供給できる

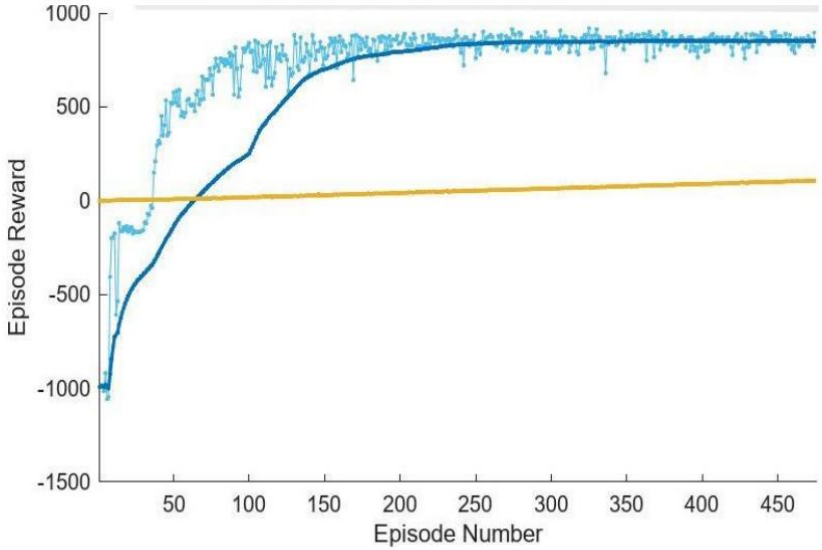
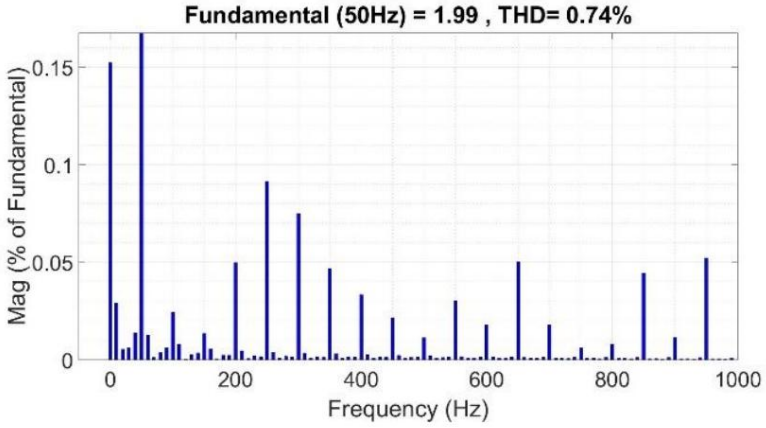
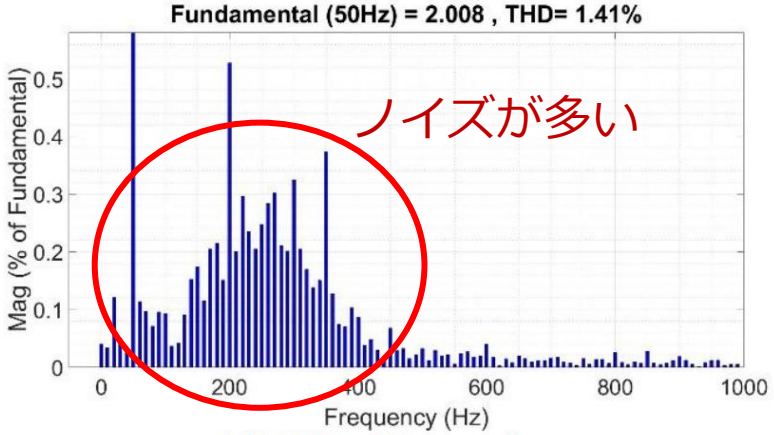


Fig. 5 Training result of DDPG algorithm



(6a) THD of RL-DDPG controller



(6b) THD of PI controller

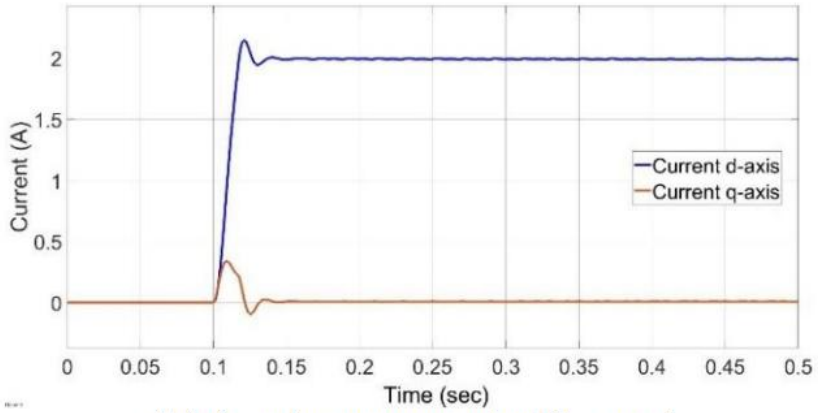
3. シミュレーション結果

■オーバーシュートとリップル

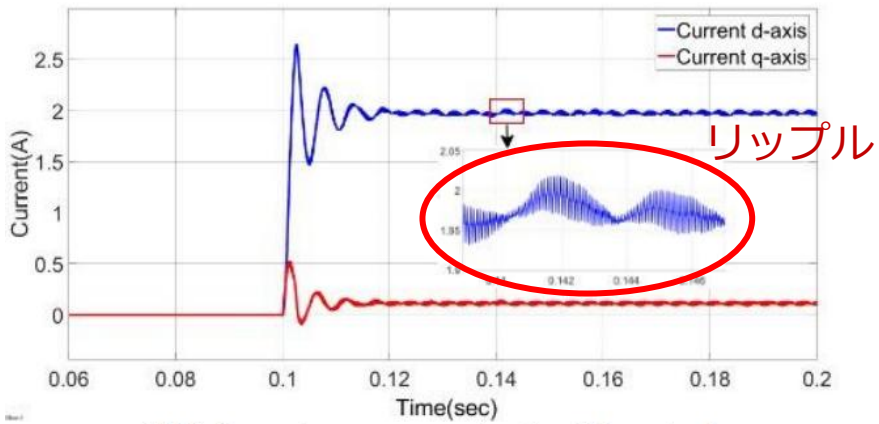
✓オーバーシュートは、DDPGは8%、PI制御は35%

✓PI制御はリップル発生

→ AI制御は、目標値に素早く・正確に到達し、リップルが少ない



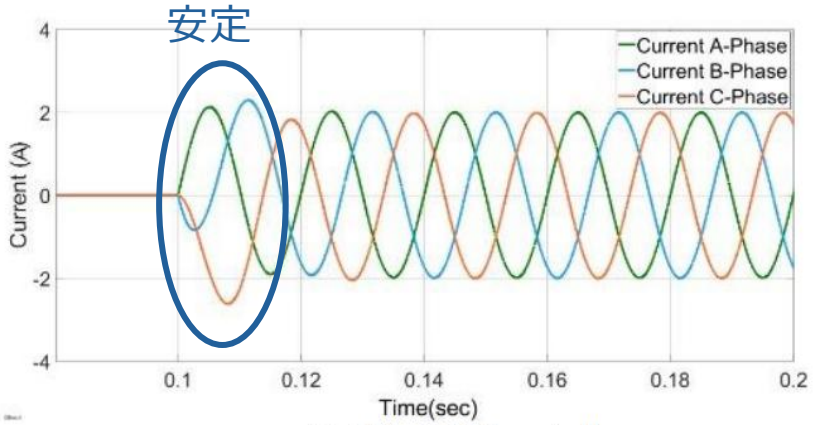
(7a) d-q axis component using RL control



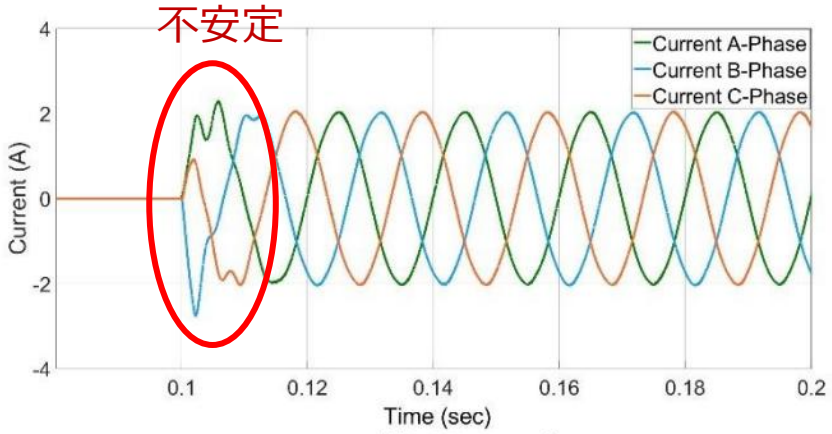
(7b) d-q axis component using PI control

■過渡期間中の応答

AI制御の方が、三相交流の波形としても乱れが少なくスムーズに変化



(8a) RL-DDPG controller



(8b) PI controller

4. 論文評価

■ DDPG制御は本当に優れているのか？

➢ PI制御に比べて、電力品質はとても良い

➢ モデルフリーなので、正確な数式モデルを事前に構築する手間を省ける

↕ けど…

- 安全面の不安

中身はブラックボックスなので数学による理論的安定性保障に劣るのではないか？

- 本当にモデルフリーなのか

「制御ゲイン調整」が「報酬関数設計」にすり替わっただけ

→ 結局、データサイエンティストや専門家の感と経験に依存するのでは？

■ 現場への実装の現実

➢ 深層学習モデルは、複雑な制御を自律的に解決する高いポテンシャルあり

↕ けど…

・リアルタイム計算のためのハードウェアコスト、時間コストを考えると、コスト面は既存制御の方が優れている

◆感想

AI制御の考えることが多すぎて、逆にPI制御が、いかに簡単で優秀なのかを知った。